The strength of this process is that it allows you to unit-test without further tangling the existing code logic: it's possible that credit_card_is_valid? depends on another three different attributes of the order, the user, or the payment system, and that's a mess you don't want to get into at this moment. The mock test lets you isolate logic issues. This style of mock-object testing also limits test coverage to the method under test, making the coverage report more accurate.

There are some problems to watch out for. It can become time-consuming to set up the external mocks for a complex method, and you run the same risk as with any mock-object testing—namely, that your test becomes a tautology because it's just parroting the input to the mock objects. You can avoid that problem by trying not to mock the data of the exact object under test.

Find the Seam

Test doubles are a specific version of a more general technique for working with legacy code, which involves finding *seams* in the code and exploiting them to make testing the legacy functionality possible

A seam is a place where you can change your application's behavior without changing the code. A test double acts as a seam because adding the test double, which happens in the test, changes the behavior of the code by mandating a specific response to a method call without executing the method. Again, the behavior of the method under test changes in the test environment without affecting behavior in production and without changing the existing development code.

It sounds magical, but the basic idea is simple and Ruby makes it easy to execute. You redirect a method call from its intended target to some other code that you want to run during tests. A test double does this by replacing the entire method call with a return value, but the generic form lets you do anything you want instead of the method call.

You might create your own object if you wanted a side effect that a mock package wouldn't normally provide, such as diagnostic logging. (Feathers calls this a *pebble*: a fake object that logs its own path through the code.) Alternatively, you might want a more elaborate processing of arguments or state than a mock can easily provide—to re-create the output of a web service your application depends on, for example. (Even if the test-double library allows you to pass an arbitrary block as the result of the stubbed call, it's often more readable to create your own object.)

Let's take some sample Ruby code that you want to test. In this sample, flit_server is an object in your system representing an internal server, and those